Informe de Resultados

Prueba de carga – Servicio “App Transaction Balance”

Dato	Valor
Fecha de ejecución	08 de octubre de 2026
Ventana de la prueba	≈ 01:35 a 02:38 (≈ 63 minutos)
Herramienta	Grafana k6 (resumen textSummary.txt + monitoreo VUs / http_reqs)
Carga aplicada	140 usuarios virtuales (VUs) concurrentes, modelo cerrado
Elaborado por	Juan Moreno – QA

1. Resumen ejecutivo

La prueba ejecutó 276.650 peticiones con 140 usuarios virtuales durante aproximadamente 63 minutos, con un throughput promedio de 73,18 peticiones por segundo. Todos los umbrales (thresholds) configurados en k6 aparecen como cumplidos (✓). Sin embargo, el análisis detallado muestra que el sistema NO tuvo un comportamiento estable:

- **Degradación severa de ≈ 9 minutos** (01:51 a 02:00): con la carga constante en 140 VUs, el throughput cayó de ≈ 80 a ≈ 10–20 peticiones/s, una reducción cercana al 85 %.
- **6.759 peticiones fallidas (2,44 %),** el 88,6 % por errores 5xx del servidor y el 99,9 % durante la etapa de carga sostenida.
- **Percentil 95 del tiempo de respuesta de 1,57 s,** por encima del límite de referencia de 1,5 s, con un máximo de 29,93 s.
- **Las peticiones fallidas tardaron en promedio ≈ 5,9 s,** ocho veces más que las exitosas (0,74 s): son fallos lentos, típicos de timeouts.

> **Veredicto: NO APROBADO.** El servicio cumple formalmente los umbrales configurados, pero presenta un episodio de inestabilidad que en producción afectaría a los usuarios durante varios minutos. Se recomienda identificar la causa raíz y repetir la prueba antes de aprobar el paso a producción.

2. Resultados principales

2.1 Volumen y throughput

Métrica	Valor	Lectura
Peticiones totales (http_reqs)	276.650	1 petición por iteración (iterations = 276.650)
Throughput promedio	73,18 req/s	Promedio de toda la prueba, incluye rampas y degradación
Throughput en estado estable	≈ 75–85 req/s	Leído del diagrama (82,6 req/s a las 02:02)
VUs máximos	140	Carga constante durante la etapa sostenida

Métrica	Valor	Lectura
Duración de la iteración (avg)	1,86 s	≈ 0,86 s de respuesta + ≈ 1 s de pausa (think time)
Datos recibidos / enviados	842 MB / 588 MB	≈ 3,0 KB recibidos y ≈ 2,1 KB enviados por petición

2.2 Tiempos de respuesta

Petición	Promedio	Mediana	p(90)	p(95)	Máximo
Todas (http_req_duration)	861,68 ms	613,42 ms	1,28 s	1,57 s	29,93 s
Exitosas (expected_response)	735,84 ms	600,70 ms	1,22 s	1,42 s	26,72 s
Fallidas (calculado*)	≈ 5,89 s	–	–	–	≈ 29,93 s

** Promedio de las fallidas = (861,68 ms × 276.650 − 735,84 ms × 269.891) ÷ 6.759 ≈ 5.887 ms.*

2.3 Desglose del tiempo de respuesta

Fase	Promedio	Lectura
http_req_waiting (servidor)	861,21 ms	99,9 % del tiempo total: el tiempo se consume en el servidor
http_req_blocked	10,97 µs	Sin espera por conexiones disponibles en el cliente
http_req_connecting / tls	3,3 µs / 7,36 µs	Conexiones reutilizadas (keep-alive); la red no es el cuello de botella
http_req_sending / receiving	43 µs / 424 µs	Envío y descarga despreciables

2.4 Errores

Etapa	Tipo de error	Cantidad	% de errores	Tasa
Stage 0 (rampa de subida)	HTTP 5xx	1	0,01 %	0,0003 /s
Stage 1 (carga sostenida)	HTTP 4xx	769	11,38 %	0,20 /s
Stage 1 (carga sostenida)	HTTP 5xx	5.987	88,58 %	1,58 /s
Stage 2 (rampa de bajada)	HTTP 5xx	2	0,03 %	0,0005 /s
Total		6.759	100 %	2,44 % del total

El total de errores por etapa (6.759) coincide con failed_request y con los checks fallidos (97,55 % de checks exitosos = 269.891 de 276.650).

3. Análisis del diagrama: usuarios virtuales vs peticiones por segundo

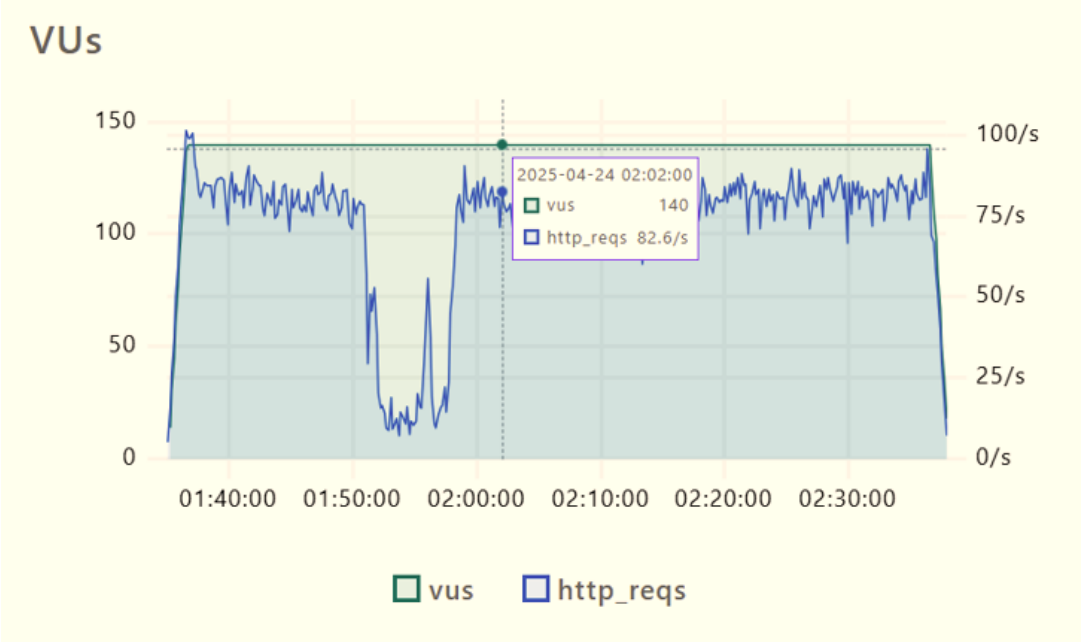


Figura 1. VUs (verde, eje izquierdo) y peticiones por segundo (azul, eje derecho) durante la prueba.

Fase	VUs	Peticiones/s	Observación
01:35 – 01:37 Rampa de subida	0 → 140	Pico de ≈ 100/s	Al inicio el sistema responde rápido y el throughput supera el nivel estable; luego se acomoda.
01:37 – 01:51 Estable	140	≈ 75 – 85/s	Comportamiento esperado: 140 VUs ÷ (≈ 0,7 s de respuesta + 1 s de pausa) ≈ 82/s.
01:51 – 02:00 Degradación	140 (constante)	≈ 10 – 20/s, mínimo ≈ 10/s	Con los mismos usuarios, el throughput cae ≈ 85 %. Hay un rebote breve a ≈ 50/s cerca de las 01:56 y una nueva caída.
02:00 – 02:37 Recuperación	140	≈ 75 – 85/s	El sistema se recupera sin cambios en la carga (82,6/s a las 02:02) y se mantiene estable hasta el final.
02:37 – 02:38 Rampa de bajada	140 → 0	→ 0	Finalización normal.

3.1 Relación entre VUs y peticiones por segundo

En un modelo cerrado como el de esta prueba, cada usuario virtual envía una petición, espera la respuesta, hace una pausa de ≈ 1 s y repite. Por la ley de Little, el throughput es:

> **Peticiones/s = VUs ÷ (tiempo de respuesta + pausa)**

- **Estado estable:** 140 ÷ (0,7 s + 1 s) ≈ 82 peticiones/s, que coincide con el diagrama.
- **Durante la degradación:** para que 140 VUs generen solo ≈ 10 peticiones/s, el tiempo de respuesta tuvo que subir a ≈ 13 s (140 ÷ 10 − 1). Esto explica el máximo de 29,93 s y por qué las peticiones fallidas promedian ≈ 5,9 s.

- **Conclusión del diagrama:** la caída del throughput no la causó un aumento de usuarios, porque los VUs se mantuvieron fijos en 140. La causó un aumento drástico del tiempo de respuesta del servidor. Mientras el tiempo de respuesta es estable, el throughput es proporcional a los VUs; cuando el servidor se degrada, esa relación se rompe.

4. Hallazgos

1. **Episodio de degradación de ≈ 9 minutos sin aumento de carga.** Entre las 01:51 y las 02:00 el throughput cayó ≈ 85 % con los VUs constantes. El sistema se recuperó solo, lo que apunta a un agotamiento temporal de recursos (pool de conexiones a base de datos, hilos, memoria/GC) o a un evento externo (dependencia lenta, proceso programado, reinicio de una instancia).
2. **Los errores son del servidor y están concentrados.** El 88,6 % de los fallos son 5xx y el 99,9 % ocurrió en la etapa de carga sostenida. Por la tasa de errores frente al throughput observado durante la caída, todo indica que la mayoría de los errores ocurrió en esa ventana de ≈ 9 minutos, donde la proporción de fallos fue muy superior al 2,44 % global.
3. **El máximo de 29,93 s sugiere timeouts de 30 s.** El timeout por defecto de k6 es 60 s, por lo que el corte cercano a 30 s probablemente lo impone el servidor, un balanceador o un API Gateway (respuestas típicas: 504 Gateway Timeout). Esto es consistente con que las peticiones fallidas tarden en promedio ≈ 5,9 s.
4. **Fallos lentos que agravan la saturación.** Una petición que espera hasta 30 s antes de fallar mantiene ocupados los recursos del servidor y empeora el problema (efecto cascada). Lo deseable es fallar rápido.
5. **El p(95) supera el límite de 1,5 s.** El p(95) de todas las peticiones es 1,57 s. Que el threshold de http_req_duration aparezca como cumplido (✓) indica que su límite configurado es más laxo que 1,5 s. Los umbrales no están alineados con el acuerdo de nivel de servicio.
6. **La tasa de error global oculta la gravedad del incidente.** El 2,44 % cumple un umbral de 3 %, pero al promediar 63 minutos se diluye un periodo en el que el servicio prácticamente no respondía. Un umbral global no detecta incidentes cortos.
7. **769 errores 4xx bajo carga.** Los errores de cliente no deberían depender de la carga. Las causas posibles son límites de tasa (429 Too Many Requests), expiración del token obtenido en la fase setup (401/403) o conflictos de datos (409). Es necesario conocer el código exacto.
8. **La red y el generador de carga no son el cuello de botella.** Los tiempos de conexión, TLS, envío y recepción son despreciables. El tiempo de espera del servidor (waiting) representa el 99,9 % del tiempo de respuesta.
9. **El modelo cerrado subestima el impacto real.** Al ponerse lento el servidor, los VUs enviaron menos peticiones y la carga sobre el sistema disminuyó justo cuando estaba saturado. En producción los usuarios siguen llegando aunque el servicio esté lento: el impacto real del incidente sería mayor (colas más largas y más errores).
10. **Horario de la prueba.** La degradación ocurrió cerca de las 01:50 de la madrugada, un horario habitual para respaldos, procesos batch, reindexación o rotación de logs. Conviene descartar que un proceso programado haya competido por recursos.

5. Conclusiones

- **Capacidad en condiciones normales:** con 140 usuarios concurrentes el servicio procesa ≈ 80 peticiones por segundo, con una mediana de ≈ 600 ms y un p(95) de las peticiones exitosas de 1,42 s.
- **Estabilidad:** el servicio no es estable bajo carga sostenida. Durante ≈ 9 minutos el throughput cayó ≈ 85 % y los tiempos llegaron hasta ≈ 30 s, sin ningún cambio en la carga.
- **Cumplimiento:** los thresholds configurados se cumplen, pero no reflejan la experiencia del usuario: el p(95) global (1,57 s) supera 1,5 s y la tasa de error de 2,44 % está cerca del límite de 3 % y concentrada en un incidente.
- **Resiliencia:** el sistema se recuperó sin intervención, lo cual es positivo, pero los fallos fueron lentos (timeouts) en lugar de rápidos, lo que prolonga la afectación.
- **Veredicto:** NO APROBADO hasta identificar y corregir la causa raíz de la degradación y obtener una nueva ejecución estable.

6. Recomendaciones

1. **Análisis de causa raíz (prioridad alta):** correlacionar la ventana 01:51–02:00 del 24/04/2025 con el monitoreo de infraestructura y APM: CPU, memoria y pausas de GC, pool de conexiones y bloqueos de la base de datos, hilos del servidor, logs del balanceador / API Gateway, eventos de autoescalado, despliegues y procesos programados.
2. **Desglosar los errores por código HTTP:** agregar a k6 contadores o tags por código de estado (500, 502, 503, 504, 401, 429…) para confirmar la hipótesis de timeouts y la causa de los 4xx.
3. **Ajustar los criterios de aceptación:** http_req_duration p(95) < 1500 ms y p(99) con un límite definido, tasa de error < 3 % con abortOnFail para detener la prueba ante fallas graves, y umbrales por etapa o por tag en lugar de solo globales.
4. **Fallar rápido:** revisar los timeouts entre el servicio y sus dependencias (bases de datos, servicios externos) y evaluar circuit breakers, para que una dependencia lenta no bloquee el servicio durante 30 s.
5. **Usar un modelo de carga abierto:** repetir la prueba con un executor de tasa constante (constant-arrival-rate) en el TPS objetivo de negocio, para medir el comportamiento real cuando el servicio se degrada.
6. **Repetir la prueba:** ejecutarla de nuevo en otro horario para verificar si el incidente se repite (aleatorio frente a sistemático) y, si se repite a la misma hora, confirmar la relación con procesos programados.
7. **Prueba de resistencia (soak):** sostener la carga varias horas para detectar si la degradación es periódica (por ejemplo, ciclos de GC o fugas de memoria o de conexiones).
8. **Prueba escalonada:** incrementar los VUs por escalones (por ejemplo, 50, 100, 140, 180) para medir la capacidad máxima y el punto en que el throughput deja de crecer.
9. **Validar el manejo del token:** si el token se obtiene una sola vez en setup, verificar que no expire durante una prueba de 63 minutos, como posible origen de los 4xx.